

CS621 HW1

Ethan Reinhart

October 2025

1

1.1 Problem

Solve the following recurrence equations using the iteration method. At the same time, you should also solve each equation using the Master Theorem if this can be done. You may assume that $T(n) = O(1)$ for $n = O(1)$.

1. $T(n) = 7T(\frac{n}{7}) + n$
2. $T(n) = T(n - 1) + 2$

1.2 Solution

1.2.1 $T(n) = 7T(\frac{n}{7}) + n$

This problem can be solved using both the Master Theorem and the iteration method.

1. Master Theorem:

$$\begin{aligned} a &= 7 \quad b = 7 \quad f(n) = n \\ \Rightarrow d &= 1 \quad \text{as } f(n) = O(n^1) \\ \Rightarrow c &= 1 \quad \text{as } c = \log_7 7 = 1 \end{aligned}$$

By this, we have $c = d$, therefore,

$$T(n) = \Theta(n^c \log_n) = \Theta(n \log_n)$$

2. Iteration Method:

$$\begin{aligned} T(n) &= 7T(\frac{n}{7}) + n \Rightarrow T(\frac{n}{7}) = 7T(\frac{n}{7^2}) + \frac{n}{7} \\ &= 7(7T(\frac{n}{7^2}) + \frac{n}{7}) + n = 49T(\frac{n}{7^2}) + 2n \Rightarrow T(\frac{n}{7^2}) = 7T(\frac{n}{7^3}) + \frac{n}{7^2} \\ &= 49(7T(\frac{n}{7^3}) + \frac{n}{7^2}) + 2n = 343T(\frac{n}{7^3}) + 3n \\ &= 7^i T(\frac{n}{7^i}) + in \end{aligned}$$

Stop iterating when $\frac{n}{7^i} = 1$ so $\log_7(n) = i$. Plus this in to get

$$\begin{aligned} T(n) &= 7^{\log_7(n)} T\left(\frac{n}{7^{\log_7(n)}}\right) + \log_7(n)n \\ &= nT\left(\frac{n}{n}\right) + \log_7(n)n \\ &= nT(1) + \log_7(n)n \\ &= \Theta(n \log(n)) \end{aligned}$$

1.2.2 $T(n) = T(n-1) + 2$

This problem can only be solved using the iteration method.

$$\begin{aligned} T(n) &= T(n-1) + 2 \Rightarrow T(n-1) = T(n-2) + 2 \\ &= T(n-2) + 4 \Rightarrow T(n-2) = T(n-3) + 2 \\ T(n) &= T(n-3) + 6 \qquad \qquad \qquad = T(n-i) + 2i \end{aligned}$$

We see $n-i = 1$ when $i = n-1$ so

$$\begin{aligned} T(n) &= T(n - (n-1)) + 2(n-1) \\ &= T(1) + 2(n-1) \\ &= \Theta(n) \end{aligned}$$

2

Design an algorithm that takes as input a positive integer n and a number x , and computes x^n by performing $O(\log n)$ multiplications.

Algorithm (Exponentiation by Squaring)

We use the binary expansion of n and square-and-multiply.

Iterative version.

```
POWER( $x, n$ ) :
  res  $\leftarrow$  1
  base  $\leftarrow x$ 
  while  $n > 0$  do
    if  $(n \bmod 2) = 1$  then res  $\leftarrow$  res  $\cdot$  base
    base  $\leftarrow$  base  $\cdot$  base
     $n \leftarrow \lfloor n/2 \rfloor$ 
  return res
```

At each iteration, $\text{base} = x^{2^k}$ for some k . Idea: Write n in "binary". Each loop step looks at the next bit. If the bit is 1, multiply the answer by the

current power; then square the power and move to the next bit. When all bits are processed, you've multiplied exactly the powers that correspond to 1-bits $\Rightarrow x^n$. Max operations will be $\log(n)$ so $\Theta(\log(n))$.

3

Given an array $A[1 \dots n]$, decide in $O(n)$ time whether there exists an element p such that more than $n/2$ elements of A equal p .

```

MAJORITY( $A$ ) :
    candidate  $\leftarrow$  null, count  $\leftarrow$  0
    for  $i = 1$  to  $n$  do
        if count = 0 then candidate  $\leftarrow A[i]$ , count  $\leftarrow$  1
        else if  $A[i] =$  candidate then count  $\leftarrow$  count + 1
        else count  $\leftarrow$  count - 1
    // verify the candidate
    occ  $\leftarrow$  0
    for  $i = 1$  to  $n$  do if  $A[i] =$  candidate then occ  $\leftarrow$  occ + 1
    return (occ >  $n/2$ , candidate)

```

During the first pass, every time we see a non-candidate element we “pair” it with one occurrence of the current candidate and decrement the count. Any perfect pairing between different elements cancels those occurrences; only a surplus of some value can survive. If an element p appears $> n/2$ times, no sequence of cancellations can remove all its occurrences, so p must be the final *candidate*. The second pass confirms whether the candidate truly appears $> n/2$ times (if no majority exists, the verification fails). Each pass is $O(n)$ time and $O(1)$ extra space. Total time $O(n)$, space $O(1)$.